ADR-003: 采用MySQL关系型数据库

状态

已采纳

背景

数据存储需求

校园二手交易平台需要存储以下核心数据:

- **用户数据**: 用户信息、管理员信息
- **商品数据**: 商品信息、商品图片
- **交易数据**: 订单信息、订单项
- **社交数据**: 评论、评价、收藏
- **管理数据**: 举报记录

数据特征

1. **关系复杂**: 用户、商品、订单之间存在复杂关联关系
2. **事务需求**: 订单创建、支付等操作需要事务保证
3. **查询多样**: 需要支持多条件查询、排序、分页
4. **数据一致性**: 需要保证数据完整性和一致性

需要解决的问题

如何选择合适的数据存储方案,以满足:

1. 数据关系管理
2. 事务支持
3. 查询性能
4. 数据一致性
5. 团队熟悉度

决策

本项目采用**MySQL 8.0.32关系型数据库**作为主数据存储。

数据库设计原则

1. **规范化设计**: 遵循第三范式,减少数据冗余
2. **合理反范式**: 在性能关键处适当冗余
3. **索引优化**: 为常用查询字段建立索引
4. **外键约束**: 保证数据引用完整性
5. **时间戳**: 所有表包含created_at和updated_at

核心表设计

用户相关表

user表: 存储用户基本信息

```
- user_id: 主键,自增
- username: 用户名,唯一
- password: 密码(加密存储)
- email: 邮箱
- phone: 手机号
- avatar_url: 头像URL
```

- campus: 校区
- address: 地址
- created_at, updated_at: 时间戳

admin表: 存储管理员信息

- admin_id: 主键,自增
- username: 用户名
- password: 密码(加密存储)
- role: 角色
- created_at: 时间戳

商品相关表

good表: 存储商品信息

- good_id: 主键,自增
- seller_id: 卖家ID(外键)
- title: 商品标题
- description: 商品描述
- price: 价格
- category: 分类
- status: 状态(在售/已售/下架)
- condition: 成色
- view_count: 浏览次数
- created_at, updated_at: 时间戳

good_image表: 存储商品图片

- image_id: 主键,自增
- good_id: 商品ID(外键)
- image_url: 图片URL
- sort_order: 排序
- created_at: 时间戳

交易相关表

order表: 存储订单信息

- order_id: 主键,自增
- buyer_id: 买家ID(外键)
- seller_id: 卖家ID(外键)
- order_number: 订单号(唯一)
- total_amount: 总金额
- status: 订单状态
- created_at, updated_at: 时间戳

order_item表: 存储订单项

- item_id: 主键,自增
- order_id: 订单ID(外键)
- good_id: 商品ID(外键)
- quantity: 数量
- unit_price: 单价
- subtotal: 小计

社交相关表

comment表: 存储评论

- comment_id: 主键,自增
- user_id: 用户ID(外键)
- good_id: 商品ID(外键)
- parent_comment_id: 父评论ID(支持回复)
- content: 评论内容
- created_at: 时间戳

rating表: 存储评价

- rating_id: 主键,自增
- user_id: 用户ID(外键)
- good_id: 商品ID(外键)

```
- score: 评分
- created_at: 时间戳
```

favorite表: 存储收藏

```
- favorite_id: 主键,自增
- user_id: 用户ID(外键)
- good_id: 商品ID(外键)
- created_at: 时间戳
```

管理相关表

report表: 存储举报

```
- report_id: 主键,自增
- reporter_id: 举报人ID(外键)
- target_user_id: 被举报用户ID(外键)
- target_good_id: 被举报商品ID(外键)
- report_type: 举报类型
- reason: 举报原因
- status: 处理状态
- handler_admin_id: 处理管理员ID(外键)
- handle_result: 处理结果
- created_at, updated_at: 时间戳
```

决策理由

1. **关系型数据**: 本项目数据关系复杂,关系型数据库天然适合
2. **事务支持**: MySQL提供完善的事务支持,保证数据一致性
3. **成熟稳定**: MySQL是最流行的开源数据库,稳定可靠
4. **团队熟悉**: 团队成员有MySQL使用经验
5. **生态完善**: MySQL工具链完善,便于开发和运维
6. **性能优秀**: 对于中小型应用,MySQL性能足够
7. **成本可控**: MySQL社区版免费,降低项目成本

后果

积极影响

1. **数据一致性**: 通过外键约束和事务保证数据完整性
2. **查询灵活**: SQL语言强大,支持复杂查询
3. **工具丰富**: MySQL Workbench、Navicat等工具便于管理
4. **文档完善**: MySQL文档和教程丰富,问题容易解决
5. **性能可控**: 通过索引优化可以获得良好性能
6. **备份简单**: MySQL提供多种备份方案

消极影响

1. **扩展性受限**: 关系型数据库水平扩展较难
2. **Schema变更**: 表结构变更需要谨慎,可能影响线上系统
3. **大数据量性能**: 单表数据量过大时性能下降
4. **对象关系映射**: 需要ORM框架进行对象关系映射

风险

1. **性能瓶颈风险**: 随着数据量增长,可能出现性能问题
    - **缓解措施**:
        - 为常用查询字段建立索引
        - 使用MyBatis Plus分页插件
        - 考虑引入Redis缓存热点数据
2. **数据安全风险**: 数据库可能面临SQL注入、数据泄露等威胁

    - **缓解措施**:
        - 使用MyBatis Plus参数化查询,防止SQL注入
        - 密码加密存储
        - 数据库访问权限控制

3. **数据一致性风险**: 高并发场景可能出现数据不一致
    - **缓解措施**:
        - 使用事务保证关键操作原子性
        - 乐观锁处理并发更新
        - 分布式锁(如果需要)

替代方案

方案A: PostgreSQL

拒绝原因:

1. 团队对MySQL更熟悉
2. MySQL社区更大,资源更丰富
3. 对于本项目,两者性能差异不明显

方案B: MongoDB

拒绝原因:

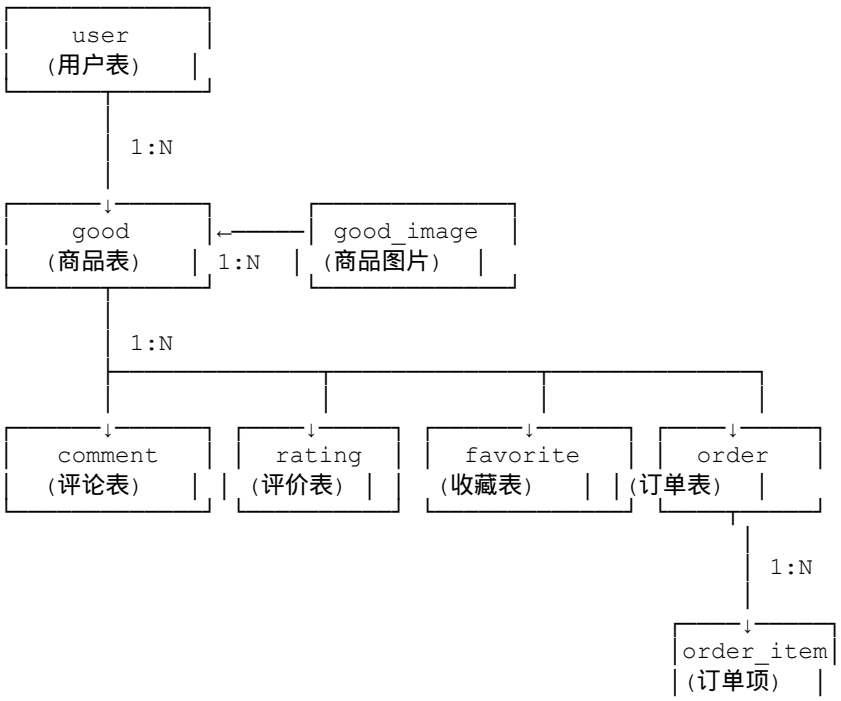
1. 本项目数据关系复杂,关系型数据库更适合
2. 需要事务支持,MongoDB事务支持相对较弱
3. 团队缺乏MongoDB经验,学习成本高

方案C: 混合存储(MySQL + Redis)

部分采纳:

- 主数据存储使用MySQL
- 考虑引入Redis作为缓存层,存储热点数据(如商品浏览数、用户会话等)
- 当前阶段先使用MySQL,后续根据性能监控决定是否引入Redis

数据库设计图



```
┌──────────────┐
│    admin     │
│ (管理员表)   │
└──────────────┘
        │
        │ 1:N
        │
        ↓
┌──────────────┐
│   report     │
│  (举报表)    │
└──────────────┘
```

索引设计

主键索引

所有表都使用自增主键,自动创建主键索引。

外键索引

所有外键字段自动创建索引:

- good.seller_id
- order.buyer_id, order.seller_id
- comment.user_id, comment.good_id, comment.parent_comment_id
- rating.user_id, rating.good_id
- favorite.user_id, favorite.good_id
- good_image.good_id
- order_item.order_id, order_item.good_id
- report.reporter_id, report.target_user_id, report.target_good_id, report.handler_admin_id

业务索引

为常用查询字段创建索引:

```
CREATE INDEX idx_user_username ON user(username);
CREATE INDEX idx_user_email ON user(email);
CREATE INDEX idx_good_seller_id ON good(seller_id);
CREATE INDEX idx_good_category ON good(category);
CREATE INDEX idx_good_status ON good(status);
CREATE INDEX idx_good_created_at ON good(created_at);
CREATE INDEX idx_order_buyer_id ON `order`(buyer_id);
CREATE INDEX idx_order_seller_id ON `order`(seller_id);
CREATE INDEX idx_order_status ON `order`(status);
CREATE INDEX idx_comment_good_id ON comment(good_id);
CREATE INDEX idx_favorite_user_id ON favorite(user_id);
```

数据库配置

字符集配置

```
DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci
```

原因:

- utf8mb4支持完整的UTF-8字符集,包括emoji
- utf8mb4_unicode_ci支持不区分大小写的Unicode排序

存储引擎

使用InnoDB存储引擎:

- 支持事务

- 支持外键
- 行级锁定
- 崩溃恢复

实施计划

阶段1: 数据库设计(第1周)

1. 完成ER图设计
2. 编写建表SQL脚本
3. 评审数据库设计

阶段2: 数据库创建(第2周)

1. 创建数据库和表
2. 创建索引
3. 插入测试数据

阶段3: ORM映射(第2-3周)

1. 使用MyBatis Plus生成Entity、Mapper
2. 编写自定义SQL
3. 测试数据访问层

阶段4: 性能优化(第7-8周)

1. 分析慢查询
2. 优化索引
3. 调整查询语句

数据库规范

命名规范

1. **表名**: 小写字母,下划线分隔,如`user`, `good_image`
2. **字段名**: 小写字母,下划线分隔,如`user_id`, `created_at`
3. **主键**: 表名单数形式 + `_id`,如`user_id`, `good_id`
4. **外键**: 关联表名单数形式 + `_id`,如`seller_id`, `buyer_id`

字段类型规范

1. **主键**: INT AUTO_INCREMENT
2. **字符串**: VARCHAR, 长度根据实际需求
3. **长文本**: TEXT
4. **金额**: DECIMAL(10, 2)
5. **时间**: TIMESTAMP
6. **状态**: VARCHAR, 存储状态描述

约束规范

1. **主键约束**: 所有表必须有主键
2. **外键约束**: 关联字段必须建立外键约束
3. **非空约束**: 必填字段设置NOT NULL
4. **默认值**: 合理设置默认值,如created_at默认当前时间
5. **唯一约束**: 需要唯一的字段设置UNIQUE,如username, order_number

参考

- MySQL官方文档: https://dev.mysql.com/doc/
- "高性能MySQL" by Baron Schwartz et al.
- "数据库系统概念" by Abraham Silberschatz et al.

决策历史

- 2026-05-01: 确定使用MySQL作为主数据库
- 2026-05-01: 完成数据库表结构设计
- 2026-05-01: 创建数据库和表